

PROJECT REPORT

PROGRAMMING IN JAVA

Library Management System

— — —

SUBMITTED BY	Sharanya Saxena
REGISTRATION NO.	24BAC10008
SUBJECT	Programming in Java
SUBMISSION TYPE	VITyarthi-Build Your Own Project
DATE	18 September 2026

TABLE OF CONTENTS

Library Management System — Project Report

2. Introduction.....3

3. Problem Statement3

4. Functional Requirements4

 4.1 – 4.3 Book, Member & Transaction Modules 4

5. Non-Functional Requirements.....5

6. System Architecture.....5

7. Design Diagrams7

 7.1 – 7.5 Use Case · Workflow · Sequence · Class · ER..... 7

8. Design Decisions & Rationale..... 10

9. Implementation Details..... 10

 9.1 – 9.4 Tech Stack · Source Files · Algorithms.....10

10. Screenshots / Results 11

11. Testing Approach..... 13

12. Challenges Faced 13

13. Learnings & Key Takeaways 14

14. Future Enhancements..... 14

15. References 14

2. Introduction

Libraries — whether in a school, college, or a small community centre — need a reliable way to track which books they own, who has borrowed them, and when they are due back. Where the collection is small, this is frequently still done with a physical register, which is slow to search, easy to misrecord, and offers no way to automatically flag overdue books or calculate fines consistently.

This project, the Library Management System, is a console-based application written in core Java that automates this process end-to-end. It was built to demonstrate the practical application of object-oriented programming, exception handling, file I/O, collections, the Java Time API, and basic security concepts (password hashing) that were covered in the Programming in Java course.

Why it matters

The system is intentionally scoped to run without any external database server or GUI framework — all data is persisted to structured CSV files on disk — so that it can be compiled and run with nothing more than a standard JDK installation, keeping the project realistic and fully reproducible for evaluation.

3. Problem Statement

Manual, register-based library management suffers from several concrete problems:

- Searching for a book by title, author, or category takes time and is prone to human error.
- There is no automatic way to know how many copies of a book are currently available.
- Due dates and overdue fines must be calculated by hand, which is inconsistent and slow.
- There is no audit trail of who accessed or changed the records.
- Member borrowing limits are not enforced systematically.

Objective

Build a Java application that lets a librarian (admin) securely log in and manage the full book/member/transaction lifecycle — adding and searching books, registering members, issuing and returning books with automatic due-date and fine calculation, and viewing a live summary of library activity — while persisting all data reliably between runs.

4. Functional Requirements

The system is organised into three major functional modules, each with a clear input/output structure and a logical user workflow:

4.1 Book Management Module

Function	Input	Output
Add Book	ISBN, title, author, category, copies	New catalogue entry created; confirmation or duplicate-ISBN error
Update Book	ISBN + new field values	Existing entry modified; error if ISBN not found
Delete Book	ISBN	Entry removed; blocked if any copies are currently issued
Search Books	Keyword (title/author/category/ISBN)	List of matching books with availability

4.2 Member Management Module

Function	Input	Output
Register Member	Name, email, phone	Auto-generated member ID (e.g. M001)
Remove Member	Member ID	Record deleted; blocked if books are still issued
View History	Member ID	Full list of that member's past and current transactions

4.3 Transaction Management Module

Function	Input	Output
Issue Book	ISBN, Member ID	Transaction ID + due date (issue date + 14 days)
Return Book	Transaction ID	Book marked returned; fine calculated automatically if overdue
View Overdue	—	List of all currently overdue transactions

Logical workflow

Admin logs in → selects a module from the main menu → performs an action → the system validates input, applies business rules, updates in-memory state, immediately persists the change to CSV, and logs the event → control returns to the menu. This “write-through” design (see Section 8) means no data is lost even if the application is closed unexpectedly.

5. Non-Functional Requirements

Requirement	How it is addressed
Security	Admin passwords are never stored or compared in plain text; they are hashed with SHA-256 (AuthService.hash()) before being written to data/admin.csv. Login is limited to 3 attempts per session.
Reliability	Every change (add/issue/return/delete) is written to disk immediately after it succeeds in memory, so the CSV files always reflect the last confirmed state even after a crash or forced exit.
Error Handling	Four custom checked exceptions (BookNotFoundException, BookNotAvailableException, MemberNotFoundException, DuplicateEntryException) model every domain failure explicitly, and a top-level try/catch in Main guarantees the application never crashes on unexpected input.
Usability	A numbered, menu-driven console interface with input validation and clear, human-readable error and confirmation messages at every step.
Logging / Monitoring	Every login attempt, issue, return, and error is timestamped and appended to data/app.log via the dedicated Logger utility, enabling later auditing.
Maintainability	The codebase is organised into four clearly separated packages — model, service, util, and exception — so each concern can be modified independently (see Section 6).
Resource Efficiency	Data is kept in lightweight in-memory HashMap/List structures for O(1)/O(n) access and only synchronised to disk on change, avoiding unnecessary I/O on every read.

6. System Architecture

The application follows a simple, layered architecture that keeps the console UI, business rules, data model, and persistence concerns cleanly separated — a common pattern for maintainable Java applications even without a formal framework.

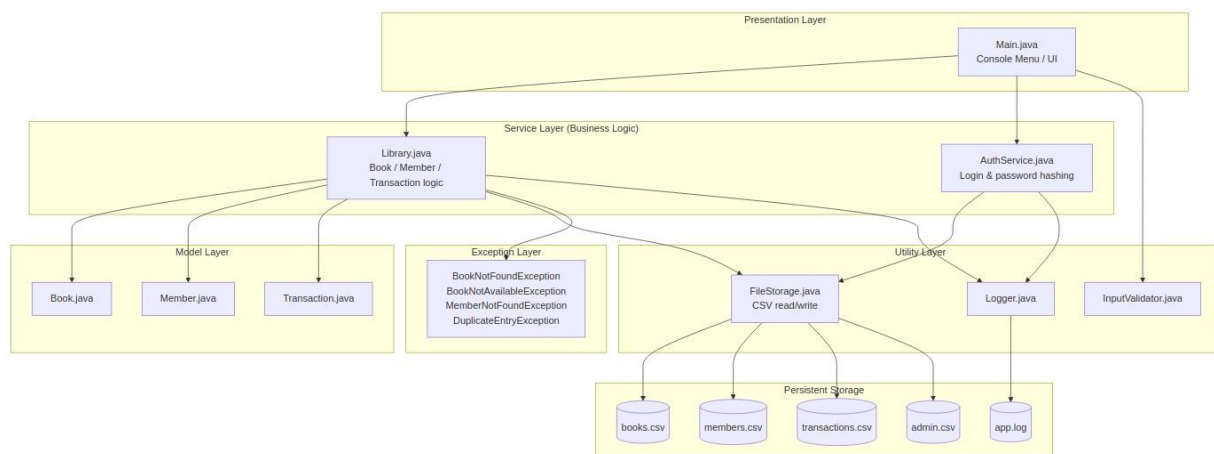


Figure 6.1 — Layered system architecture of the Library Management System

Layer	Responsibility	Classes
Presentation	Console menus, user input/output	Main
Service	Business rules, orchestrates model + utility classes	Library, AuthService
Model	Plain data objects representing domain entities	Book, Member, Transaction
Utility	Cross-cutting concerns: persistence, validation, logging	FileStorage, InputValidator, Logger
Exception	Domain-specific checked exceptions	BookNotFoundException, BookNotAvailableException, MemberNotFoundException, DuplicateEntryException

7. Design Diagrams

7.1 Use Case Diagram

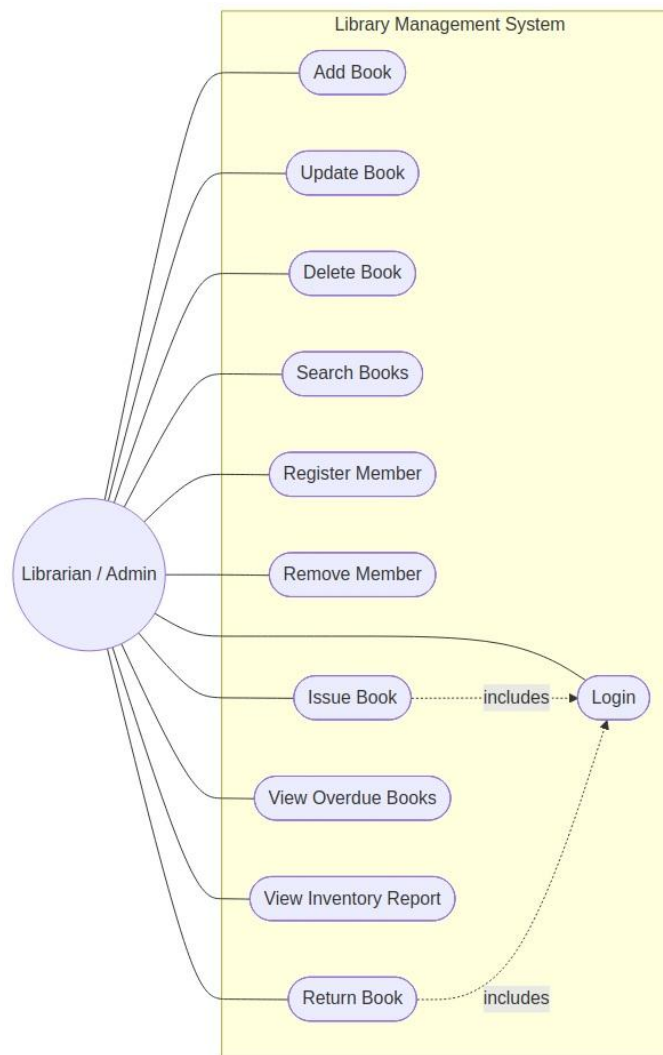


Figure 7.1 — Use cases available to the Librarian/Admin actor

7.2 Workflow Diagram (Issue / Return process)

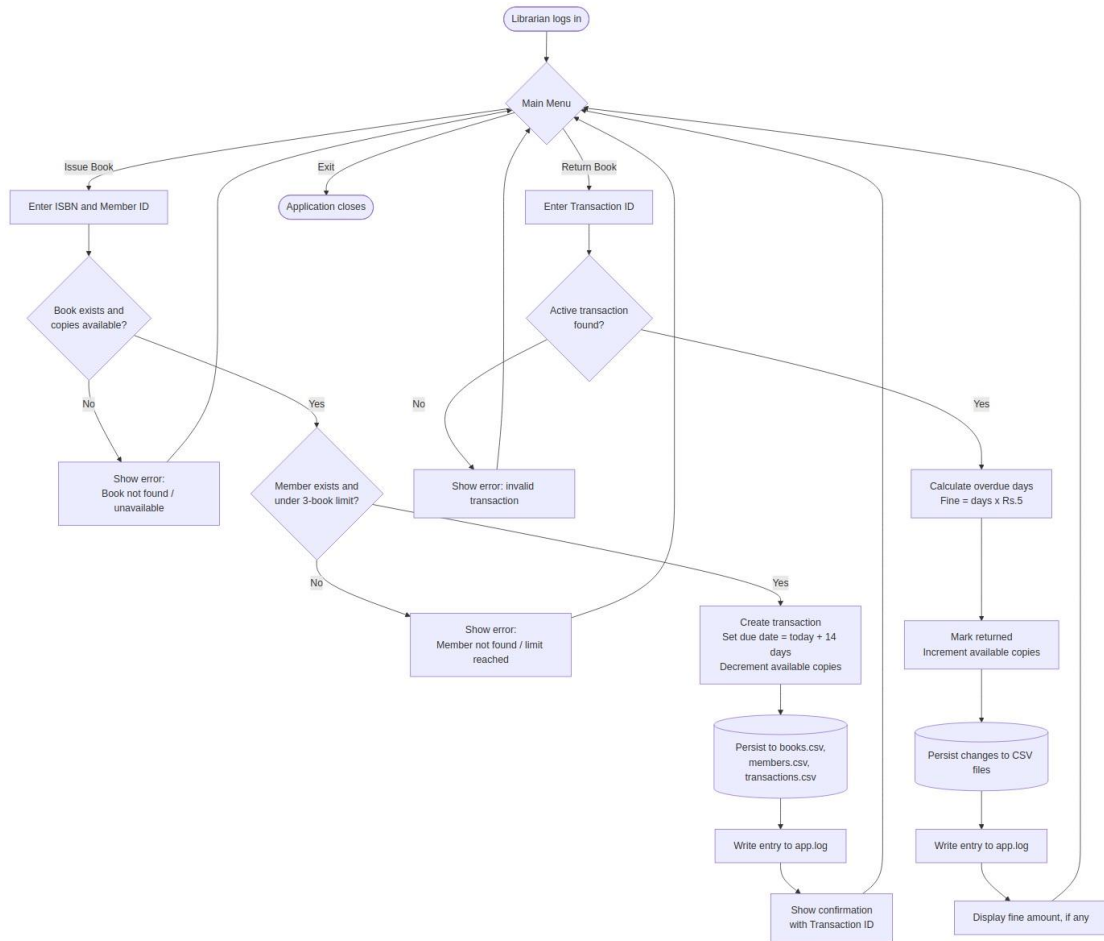


Figure 7.2 — End-to-end workflow for issuing and returning a book

7.3 Sequence Diagram (Issue Book)

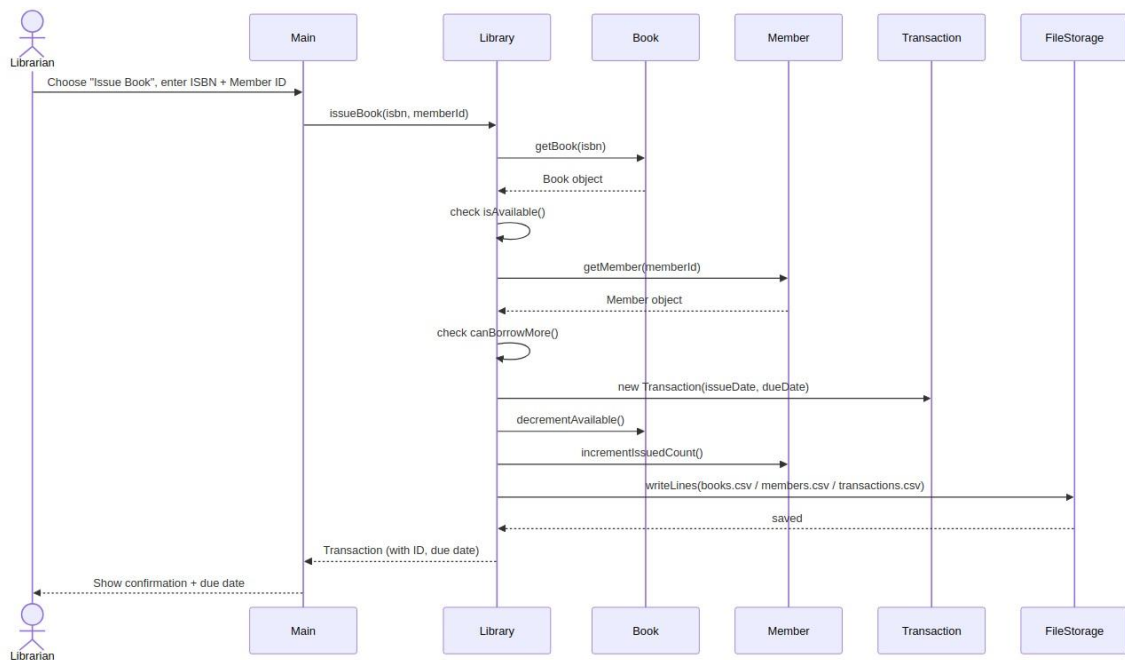


Figure 7.3 — Sequence of interactions when a book is issued

7.4 Class Diagram

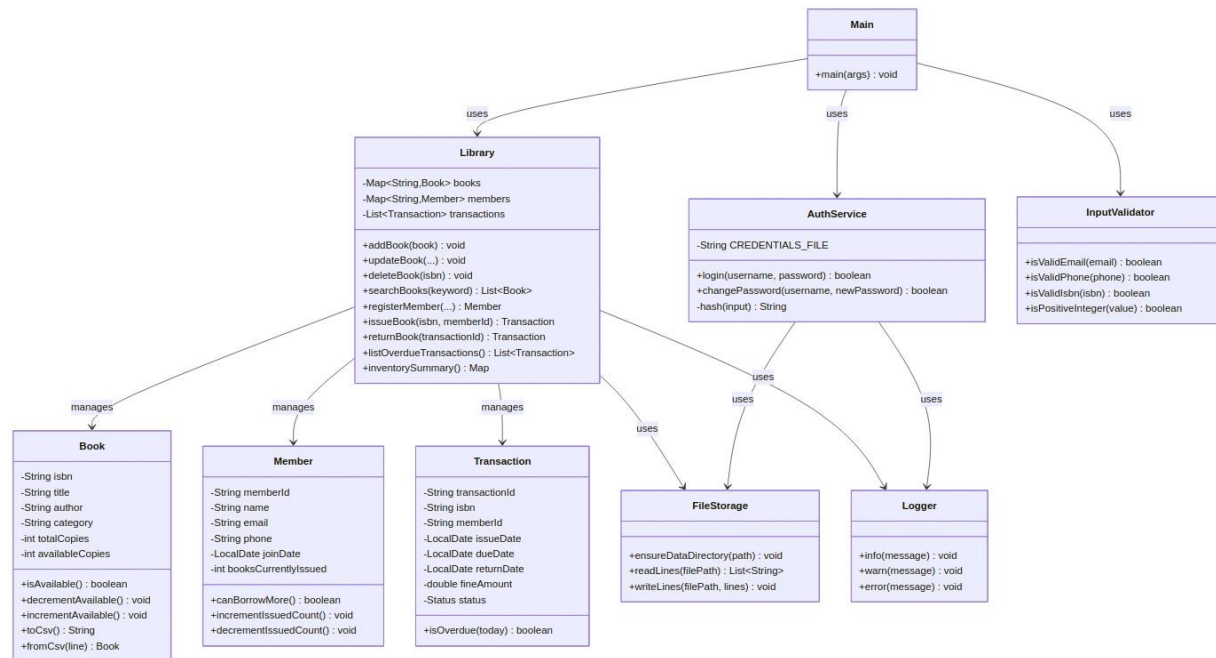


Figure 7.4 — Class relationships across the four packages

7.5 Entity-Relationship (Storage) Diagram

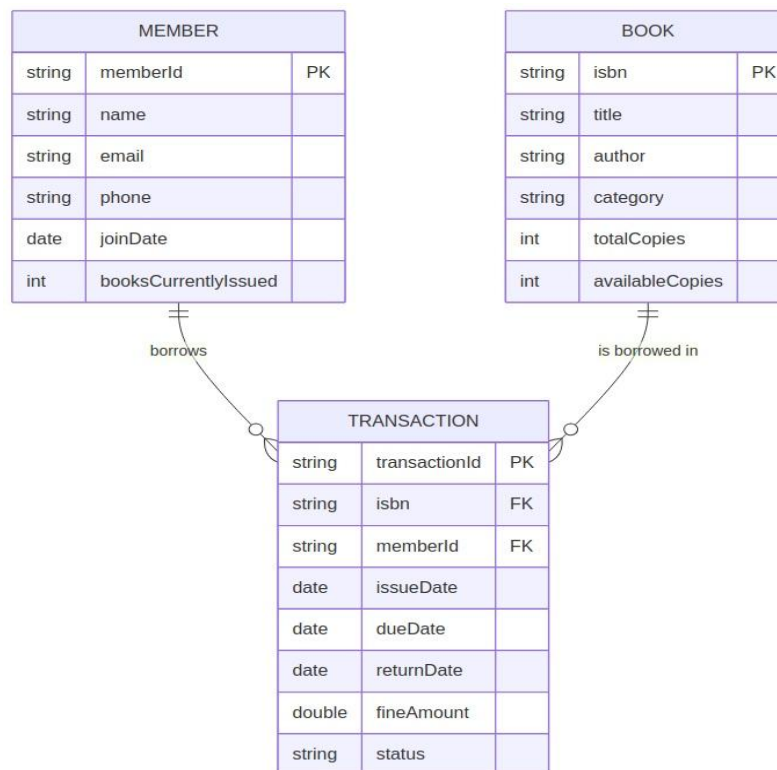


Figure 7.5 — Logical entity relationships across the three CSV “tables”

Although the project uses flat CSV files rather than a relational database, the three record types map cleanly onto an ER model: a Member can have many Transactions, and a Book can appear in many Transactions — the classic many-to-many relationship between books and members, resolved through the Transaction entity.

8. Design Decisions & Rationale

Decision	Rationale
CSV flat-file storage instead of a database (JDBC/MySQL)	Keeps the project self-contained and runnable with only a JDK, matches the scope of a course assignment, and makes the data human-readable for evaluation/debugging.
Write-through persistence (save after every change)	Simpler and more reliable than a “save on exit” model — no risk of losing data if the console window is closed abruptly.
Custom checked exceptions instead of returning null/boolean	Forces every calling method to explicitly handle failure cases at compile time, which is a core Java exception-handling concept and produces clearer error messages for the user.
SHA-256 password hashing	Even in a small course project, storing plain-text passwords is bad practice; hashing demonstrates a basic, correct security habit using only the standard <code>java.security</code> package.
Package-by-layer structure (model/service/util/exception)	A conventional Java project layout that keeps related classes together and makes the codebase easy for a new reader (or evaluator) to navigate.
3-book borrowing limit and 14-day loan period as constants	Encodes realistic library policy as simple, easily-adjustable constants in <code>Library.java</code> rather than hard-coding logic in multiple places.

9. Implementation Details

9.1 Technology Stack

- Language: Core Java (JDK 17+), no external libraries or frameworks
- Persistence: `java.io` / `java.nio.file` for CSV read/write
- Date handling: `java.time.LocalDate` and `ChronoUnit` for due-date and fine calculations
- Security: `java.security.MessageDigest` (SHA-256) for password hashing
- Collections/Streams: `HashMap`, `ArrayList`, and the Stream API for search/filter/report operations

9.2 Source File Summary (9 files)

File	Package	Purpose
Main.java	main	Console menu system and application entry point
Book.java	model	Book entity + CSV (de)serialisation
Member.java	model	Member entity + borrowing-limit rule
Transaction.java	model	Issue/return record + overdue check

File	Package	Purpose
Library.java	service	Core business logic for all three modules
AuthService.java	service	Admin login and password hashing
FileStorage.java	util	Generic CSV read/write persistence helper
InputValidator.java	util	Regex-based validation for ISBN, email, phone, numbers
Logger.java	util	Timestamped append-only logging to app.log

Four additional exception classes (BookNotFoundException, BookNotAvailableException, MemberNotFoundException, DuplicateEntryException) live in the exception package, bringing the total to 13 source files.

9.3 Key Algorithm — Fine Calculation

```
long overdueDays = Math.max(0, ChronoUnit.DAYS.between(txn.getDueDate(), today));
double fine = overdueDays * FINE_PER_DAY; // FINE_PER_DAY = Rs. 5.0
```

9.4 Key Algorithm — Unique ID Generation

Member IDs (M001, M002, ...) and Transaction IDs (T0001, T0002, ...) are generated from an in-memory sequence counter that is re-synchronised from the highest existing ID found in the CSV files at start-up, so IDs remain unique and sequential even across multiple runs of the program.

10. Screenshots / Results

Below is an actual console session captured while running the compiled program (./build.sh then ./run.sh), exercising a typical workflow: login, adding a book, registering a member, issuing the book, and returning it late to demonstrate fine calculation.

```

sharanya@lms-dev: ~/LibraryManagementSystem

sharanya@lms-dev:~/LibraryManagementSystem$ ./run.sh
=====
LIBRARY MANAGEMENT SYSTEM (Java)
=====
Username: admin
Password: *****
Login successful. Welcome, admin!

----- MAIN MENU -----
1. Book Management
2. Member Management
3. Issue / Return Books
4. Reports
0. Exit
Choose an option: 1

--- Book Management ---
1. Add book
2. Update book
3. Delete book
4. Search books
5. List all books
0. Back
Choose an option: 1
ISBN: JAVA-101
Title: Effective Java
Author: Joshua Bloch
Category: Programming
Number of copies: 3
Book added successfully.

----- MAIN MENU -----
1. Book Management
2. Member Management
3. Issue / Return Books
4. Reports
0. Exit
Choose an option: 2

--- Member Management ---
1. Register member
2. Remove member
3. List all members
4. View member's transaction history
0. Back
Choose an option: 1
Name: Rahul Verma
Email: rahul.verma@email.com
Phone (10 digits): 9876543210
Member registered with ID: M001

----- MAIN MENU -----
1. Book Management
2. Member Management
3. Issue / Return Books
4. Reports
0. Exit
Choose an option: 3

--- Issue / Return Books ---
1. Issue a book
2. Return a book
3. List overdue books
4. List all transactions
0. Back
Choose an option: 1
ISBN: JAVA-101
Member ID: M001
Book issued. Transaction ID: T0001 | Due date: 2026-10-02

... 5 days after the due date ...

----- MAIN MENU -----
1. Book Management
2. Member Management
3. Issue / Return Books
4. Reports
0. Exit
Choose an option: 3

--- Issue / Return Books ---
1. Issue a book
2. Return a book
3. List overdue books
4. List all transactions
0. Back
Choose an option: 2
Transaction ID: T0001
Book returned. Fine due: Rs.25.00 (returned 5 day(s) late)

----- MAIN MENU -----
1. Book Management
2. Member Management
3. Issue / Return Books
4. Reports
0. Exit
Choose an option: 4

--- Inventory & Library Report ---
Total titles      : 1
Total copies     : 3
Copies available  : 3
Copies issued out : 0
Registered members : 1
Overdue transactions : 0

```

Figure 10.1 — Live terminal session: ./run.sh, com.library.main.Main

Note

Screenshot captured from an actual terminal session after building the project with ./build.sh and running it with ./run.sh, using the exact input sequence shown above.

11. Testing Approach

Testing was performed as structured manual/functional validation of every module, covering both the “happy path” and the failure/edge cases that the custom exceptions are designed to catch:

#	Test Case	Expected Result
1	Login with wrong password 3 times	Application exits after the 3rd failed attempt
2	Add a book with an ISBN that already exists	DuplicateEntryException shown, no duplicate created
3	Register a member with an invalid email/phone	Validation error shown, no member created
4	Issue a book that has 0 available copies	BookNotAvailableException shown
5	Issue a 4th book to a member at the 3-book limit	Request rejected with a limit-reached message
6	Return a book after its due date	Fine = overdue days × Rs. 5 calculated and displayed
7	Delete a book that still has copies issued	Deletion blocked with an explanatory error
8	Restart the application after adding data	All books/members/transactions reload correctly from CSV
9	Search books with a partial keyword	All books whose title/author/category/ISBN contain the keyword are returned
10	Check data/app.log after a session	Every login, issue, return and error appears with a timestamp

This checklist is also included in README.md so the evaluator can reproduce each test directly against the compiled program.

12. Challenges Faced

- Keeping IDs unique across restarts: since there is no database auto-increment, the sequence counters for member and transaction IDs had to be re-derived from the maximum ID already present in the CSV files every time the program starts.
- Designing exceptions that are genuinely useful: it was tempting to use one generic exception everywhere; splitting failures into four specific, checked exception types took more upfront design but made the calling code in Main far easier to read and handle correctly.
- CSV as a “poor man's database”: commas inside free-text fields (e.g. a book title) could break the CSV format, so a simple sanitisation step (replacing commas with semicolons on write) was added to each model's toCsv() method.
- Balancing simplicity with realism: deciding how much business logic (borrowing limits, fine rates, loan periods) to hard-code as constants versus make configurable, while keeping the project within the scope of a course assignment.

13. Learnings & Key Takeaways

- Practical experience applying core OOP principles (encapsulation, single-responsibility classes) to a multi-module real-world problem rather than an isolated exercise.
- Hands-on use of Java's checked-exception mechanism to model domain failures explicitly, rather than relying on return codes.
- Working with the modern java.time API for date arithmetic (due dates, overdue-day calculation) instead of the legacy Date/Calendar classes.
- Understanding why “write-through” persistence and defensive validation matter for a reliable, crash-resistant application.
- Basic but correct use of cryptographic hashing (SHA-256) for credential storage, and why plain-text passwords should never be used, even in a small project.
- Appreciating the value of a layered package structure for maintainability as a codebase grows beyond a single file.

14. Future Enhancements

- Replace CSV storage with a real relational database (e.g. MySQL via JDBC) to support concurrent multi-user access.
- Build a JavaFX or web-based (Spring Boot) front-end for a graphical user experience.
- Add a self-service member portal so members can search the catalogue and view their own history without librarian assistance.
- Integrate email/SMS reminders a few days before a book's due date.
- Add barcode/ISBN scanning support for faster book entry and issue/return at a physical desk.
- Introduce automated unit tests (JUnit) for the business-rule methods in Library.java.

15. References

- Oracle Java SE 17 Documentation — java.io, java.nio.file, java.time, java.security packages.
- Bloch, J. — Effective Java (3rd Edition) — principles referenced for class design and encapsulation.
- Mermaid.js Documentation — used to generate the architecture, workflow, use case, class, sequence and ER diagrams in this report.
- VITyarthi — “Build Your Own Project: General Project Instructions & Submission Guidelines” (course brief followed for this submission).